

OpsFlow 核心流程详解

FlowEngine/PipelineBuilder/PluginService/信号/Celery/超时/回滚/Webhook 全流程源码级解构

2026-06-03

Generated: 2026-06-03 | Source: backend/opsflow/doc/

严格基于 [backend/opsflow/core/]、[signals/]、[tasks.py]、[consumers.py] 全部源代码生成 最后更新于: 2026-06-04

一、完整执行链

前端 DesignCanvas → 保存模板 → 发布版本 → 创建执行(冻结快照) → 启动
→ Celery Worker → FlowEngine.run() → build_bamboo_pipeline()
→ api.run_pipeline() → bamboo-engine DAG 遍历 → PluginService 路由
→ BasePlugin 执行 → post_set_state 信号 → 状态追踪/轨迹/日志/WS

二、FlowEngine 引擎

[FlowEngine] 是核心执行引擎，封装所有与 bamboo-engine 的交互。

****属性:****

• [_runtime] —

****外部 API:****

方法	触发	说明
----	----	----

方法	触发	说明
`start(sync=False)`	执行启动	status=running → 派发 Celery 任务
`pause()`	暂停	api.pause_pipeline()
`resume()`	恢复	api.resume_pipeline()
`retry(node_id)`	重试节点	api.retry_node()
`skip(node_id)`	跳过节点	api.skip_node()
`force_fail(node_id)`	强制失败	api.forced_fail_activity()
`cancel()`	取消	api.revoke_pipeline()

****内部帮助方法：****

- `[_update_node_`
- `[_fail_execut`
- `[_rollback_fai`

****run() 执行步骤：****

1. 读取 `[template_s`

三、Pipeline Builder

`[build_bamboo_pipeline()]` 将前端 `[{nodes, edges}]` 转换为 bamboo-engine 格式。

****转换步骤：****

1. `[_filter_nodes_`

****节点映射：****

node_type	bamboo-engine 元素	说明
start_event	EmptyStartEvent	视觉起点
end_event	EmptyEndEvent	视觉终点
atom	ServiceActivity	opsflow_plugin 组件
exclusive_gateway	ExclusiveGateway	条件分支
parallel_gateway	ParallelGateway	并行扇出

node_type	bamboo-engine 元素	说明
converge_gateway	ConvergeGateway	汇聚
subprocess	SubProcess	子流程

四、PluginService 路由

[OpsflowPluginComponent] → [PluginService]:

- **execute()**: 提取 [_atom_type] → [get_plugin()] → 变量解析 → [plugin_cls().execute()] → [_promote_result()]
- **schedule()**: 异步插件轮询回调
- **rollback()**: 失败时回滚调用

五、信号系统

[post_set_state] 信号 → [on_post_set_state()]:

- 根节点 FINISHED → [_handle_root_state_change()] → 更新 execution.status
- 子节点变更 → [_safe_signal_handler] 循环处理:
- [_update_execution_node_status()] — 原子 JSON_SET 更新
- [_update_state_tree()] — 增量更新
- [_record_node_trace()] — 轨迹记录
- [_update_node_timeout()] — Redis 超时追踪

六、Celery 任务

任务	队列	说明
execute_pipeline_task	er_execute	主执行入口
notify_node_status	er_execute	WS 节点状态推送
notify_execution_completed	er_execute	WS 完成通知
auto_retry_node_task	er_execute	自动重试
execute_node_timeout_strategy	er_execute	超时策略执行
webhook_send	er_execute	Webhook 回调
retry_schedule_execution	er_execute	调度重试

七、状态机

NodeState: PENDING → RUNNING → FINISHED(终态)/FAILED → RETRY

PipelineState:

八、自动重试+超时

- [AutoRetryStrategyCreator.batch_create_strategy()] — 执行前创建策略
- [dispatch_auto_retry()] — FAILED 时检查策略→派发 Celery 重试

- [NodeTimeoutStrategy.dispatch_timeout_nodes()] — Redis ZRANGEBYSCORE 扫描到期节点

九、WebSocket

[FlowMonitorConsumer(AsyncJsonWebsocketConsumer)] — 路径 [ws/opsflow/execution/{id}/]

消息类型: [init_state] / [node_status] / [tower_job_update] / [execution_completed]